

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

from tensorflow.keras import layers, models

print("TensorFlow version:", tf.__version__)
print("Libraries imported successfully!")
```

TensorFlow version: 2.20.0
Libraries imported successfully!

```
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()
```

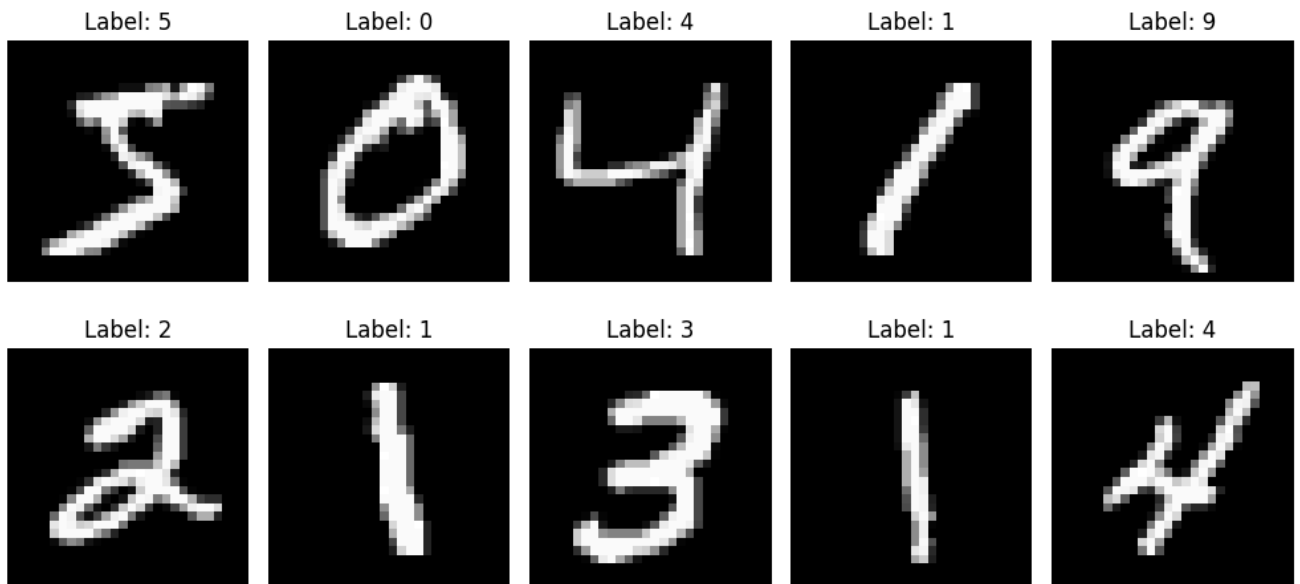
```
print("Training images:", x_train.shape)
print("Training labels:", y_train.shape)
print("Testing images:", x_test.shape)
print("Testing labels:", y_test.shape)
```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz>
11490434/11490434 — 2s 0us/step
Training images: (60000, 28, 28)
Training labels: (60000,)
Testing images: (10000, 28, 28)
Testing labels: (10000,)

```
plt.figure(figsize=(10, 5))

for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(x_train[i], cmap="gray")
    plt.title(f"Label: {y_train[i]}")
    plt.axis("off")

plt.tight_layout()
plt.show()
```



```
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

x_train = np.expand_dims(x_train, axis=-1)
x_test = np.expand_dims(x_test, axis=-1)

print("Training data shape:", x_train.shape)
print("Testing data shape:", x_test.shape)
```

Training data shape: (60000, 28, 28, 1)
Testing data shape: (10000, 28, 28, 1)

```

model = models.Sequential([

    layers.Conv2D(
        32,
        (3, 3),
        activation="relu",
        input_shape=(28, 28, 1)
    ),

    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(
        64,
        (3, 3),
        activation="relu"
    ),

    layers.MaxPooling2D((2, 2)),

    layers.Flatten(),

    layers.Dense(128, activation="relu"),

    layers.Dense(10, activation="softmax")
])

model.summary()

```

/usr/local/lib/python3.13/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input\_shape`
 super().__init__(activity_regularizer=activity_regularizer, **kwargs)

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dense (Dense)	(None, 128)	204,928
dense_1 (Dense)	(None, 10)	1,290

Total params: 225,034 (879.04 KB)

Trainable params: 225,034 (879.04 KB)

Non-trainable params: 0 (0.00 B)

```

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

print("Model compiled successfully!")

```

Model compiled successfully!

```

history = model.fit(
    x_train,
    y_train,
    epochs=5,
    batch_size=64,
    validation_split=0.2
)

```

```

Epoch 1/5
750/750 ————— 46s 61ms/step - accuracy: 0.9934 - loss: 0.0203 - val_accuracy: 0.9901 - val_loss: 0.0369
Epoch 2/5
750/750 ————— 46s 61ms/step - accuracy: 0.9952 - loss: 0.0145 - val_accuracy: 0.9872 - val_loss: 0.0495
Epoch 3/5
750/750 ————— 80s 59ms/step - accuracy: 0.9960 - loss: 0.0126 - val_accuracy: 0.9894 - val_loss: 0.0397
Epoch 4/5
750/750 ————— 83s 61ms/step - accuracy: 0.9966 - loss: 0.0102 - val_accuracy: 0.9913 - val_loss: 0.0352

```

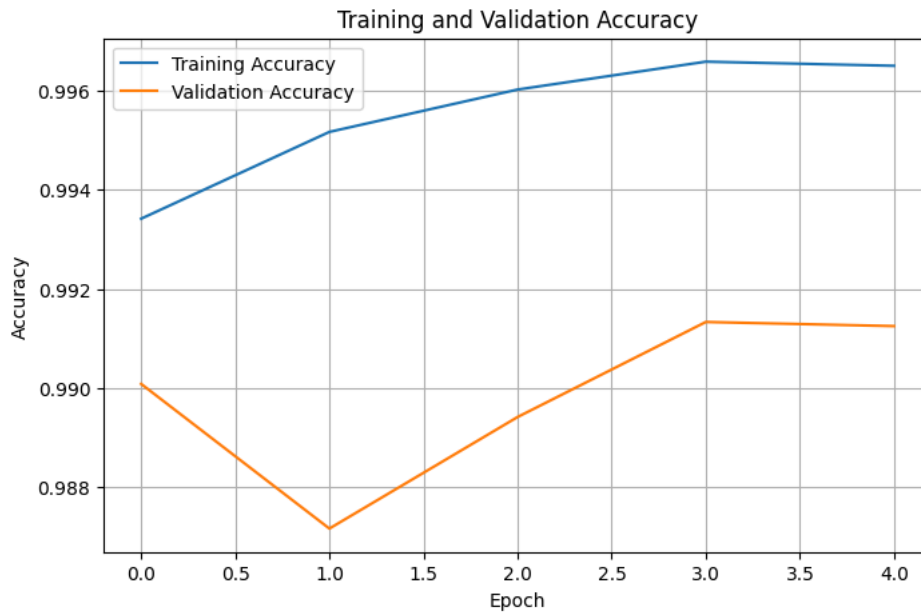
Epoch 5/5
 750/750 ————— 80s 57ms/step - accuracy: 0.9965 - loss: 0.0096 - val_accuracy: 0.9912 - val_loss: 0.0413

```
plt.figure(figsize=(8, 5))

plt.plot(history.history["accuracy"], label="Training Accuracy")
plt.plot(history.history["val_accuracy"], label="Validation Accuracy")

plt.title("Training and Validation Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)

plt.show()
```



```
test_loss, test_accuracy = model.evaluate(
    x_test,
    y_test,
    verbose=0
)
```

```
print("Test Loss:", test_loss)
print("Test Accuracy:", test_accuracy)
```

Test Loss: 0.0319230780005455
 Test Accuracy: 0.9897000193595886

```
from sklearn.metrics import precision_score, recall_score, f1_score
```

```
# Predict classes for test images
y_pred_prob = model.predict(x_test, verbose=0)
y_pred = np.argmax(y_pred_prob, axis=1)
```

```
# Calculate metrics
precision = precision_score(y_test, y_pred, average="weighted")
recall = recall_score(y_test, y_pred, average="weighted")
f1 = f1_score(y_test, y_pred, average="weighted")
```

```
print(f"Precision : {precision:.4f}")
print(f"Recall : {recall:.4f}")
print(f"F1-Score : {f1:.4f}")
```

Precision : 0.9897
 Recall : 0.9897
 F1-Score : 0.9897

```
from sklearn.metrics import confusion_matrix
import seaborn as sns
```

```

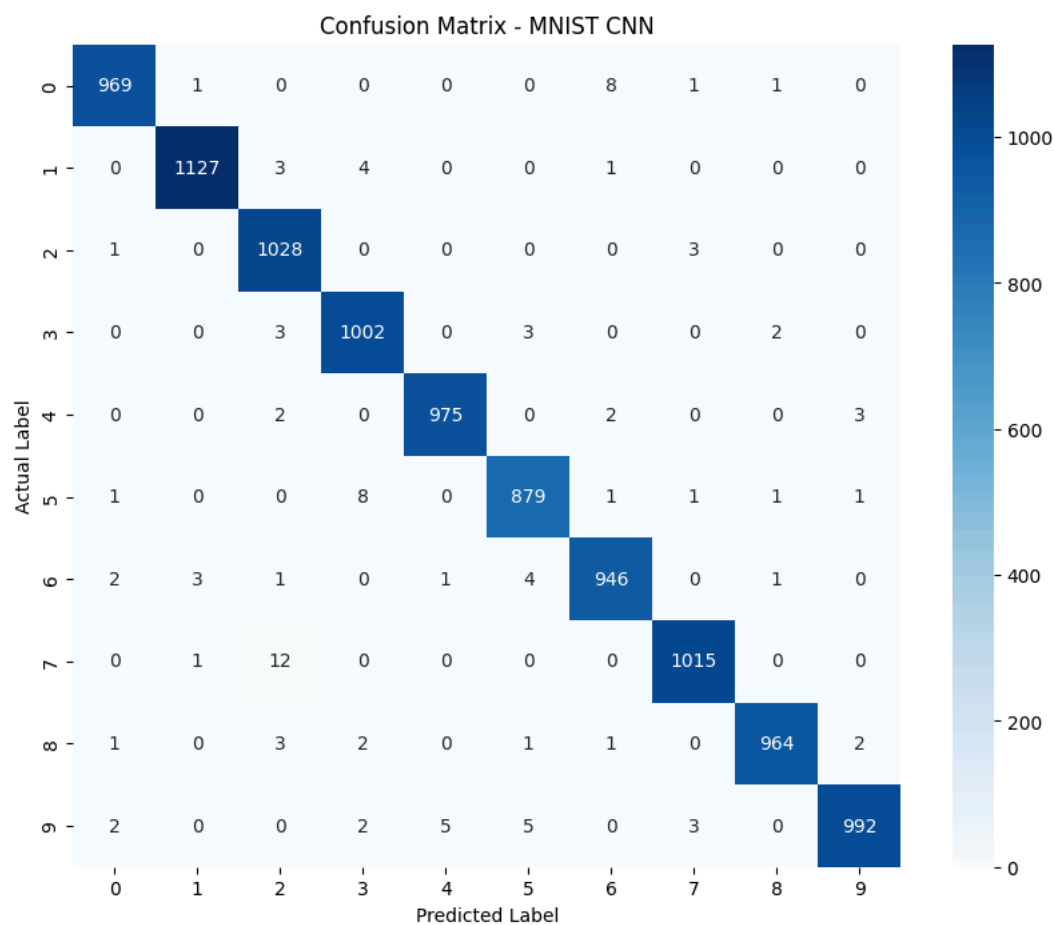
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(10, 8))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=range(10),
    yticklabels=range(10)
)

plt.title("Confusion Matrix - MNIST CNN")
plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")
plt.show()

```



```

index = 0

image = x_test[index]
actual_label = y_test[index]

prediction = model.predict(
    np.expand_dims(image, axis=0),
    verbose=0
)

predicted_label = np.argmax(prediction)

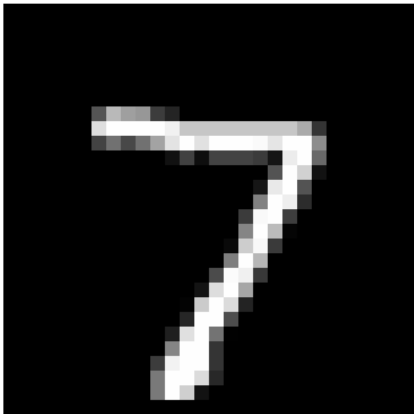
plt.figure(figsize=(4, 4))
plt.imshow(image.squeeze(), cmap="gray")
plt.title(
    f"Actual: {actual_label} | Predicted: {predicted_label}"
)
plt.axis("off")
plt.show()

print("Actual Label:", actual_label)

```

```
print("Predicted Label:", predicted_label)
```

Actual: 7 | Predicted: 7



Actual Label: 7
Predicted Label: 7

```
plt.figure(figsize=(12, 6))

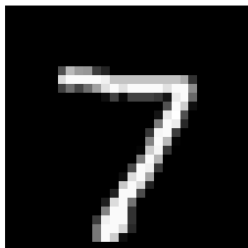
for i in range(10):
    prediction = model.predict(
        np.expand_dims(x_test[i], axis=0),
        verbose=0
    )

    predicted_label = np.argmax(prediction)

    plt.subplot(2, 5, i + 1)
    plt.imshow(x_test[i].squeeze(), cmap="gray")
    plt.title(
        f"Actual: {y_test[i]}\nPredicted: {predicted_label}"
    )
    plt.axis("off")

plt.tight_layout()
plt.show()
```

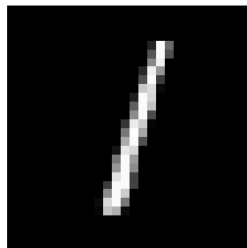
Actual: 7
Predicted: 7



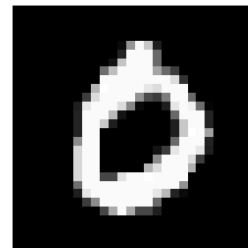
Actual: 2
Predicted: 2



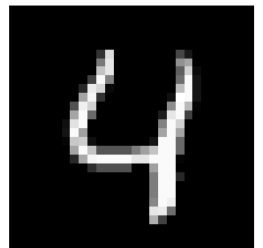
Actual: 1
Predicted: 1



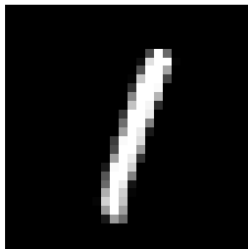
Actual: 0
Predicted: 0



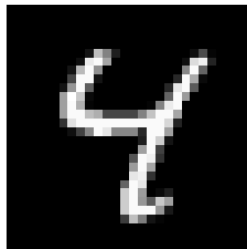
Actual: 4
Predicted: 4



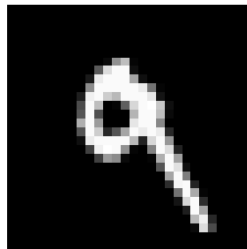
Actual: 1
Predicted: 1



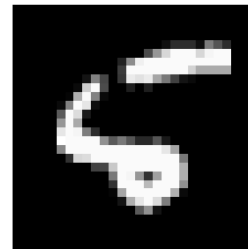
Actual: 4
Predicted: 4



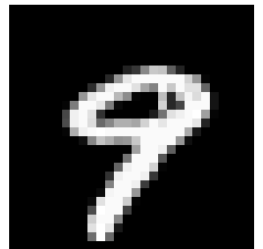
Actual: 9
Predicted: 9



Actual: 5
Predicted: 5



Actual: 9
Predicted: 9



```
model.save("mnist_cnn_model.keras")
```

```
print("CNN model saved successfully!")
```

CNN model saved successfully!